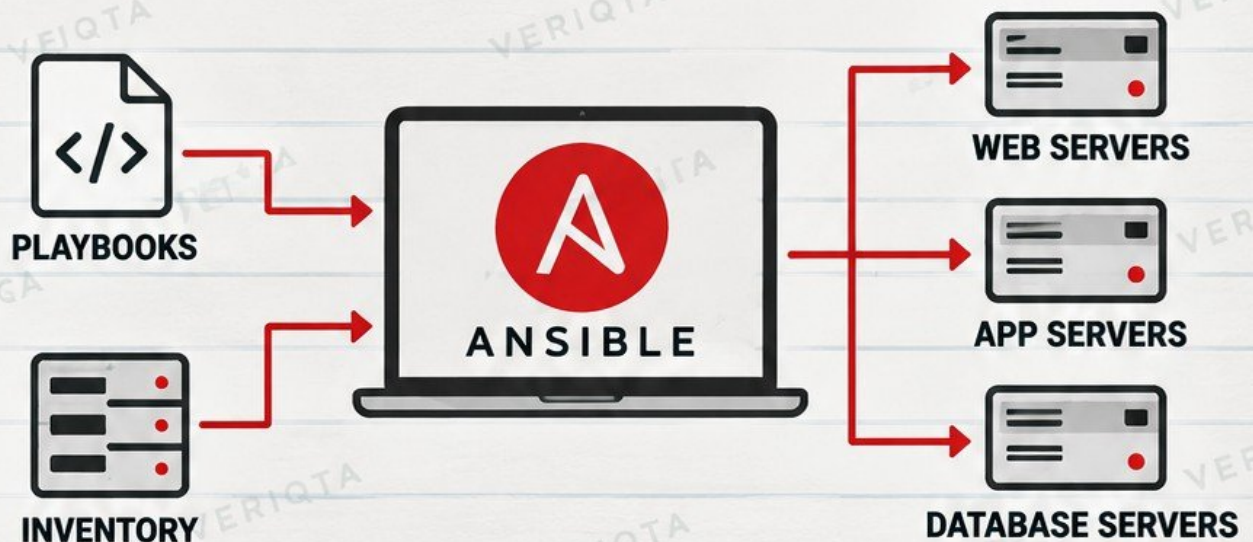




Ansible

Crash Course



**AUTOMATE
CONFIGURATION**



**MANAGE
SERVERS**



**DEPLOY
APPLICATIONS**



**IMPROVE
CONSISTENCY**



**BUILD
REAL SKILLS**

01 Ansible in 5 Minutes

What Ansible is, why configuration management exists, and where Ansible fits in DevOps

What is Ansible?

- **Ansible is an open-source automation tool** used to configure, manage and deploy software on multiple servers.
- It uses simple YAML files called playbooks to describe what you want your servers to do.
- It is **agentless**, meaning you do not need to install any software on the servers you manage.

Why Does Configuration Management Exist?

- Manually configuring servers is **slow**, **error-prone** and **not consistent**.
- Configuration management ensures **all servers are configured the same way, every time**.
- It helps you automate repetitive tasks such as installing packages, configuring files and managing services.

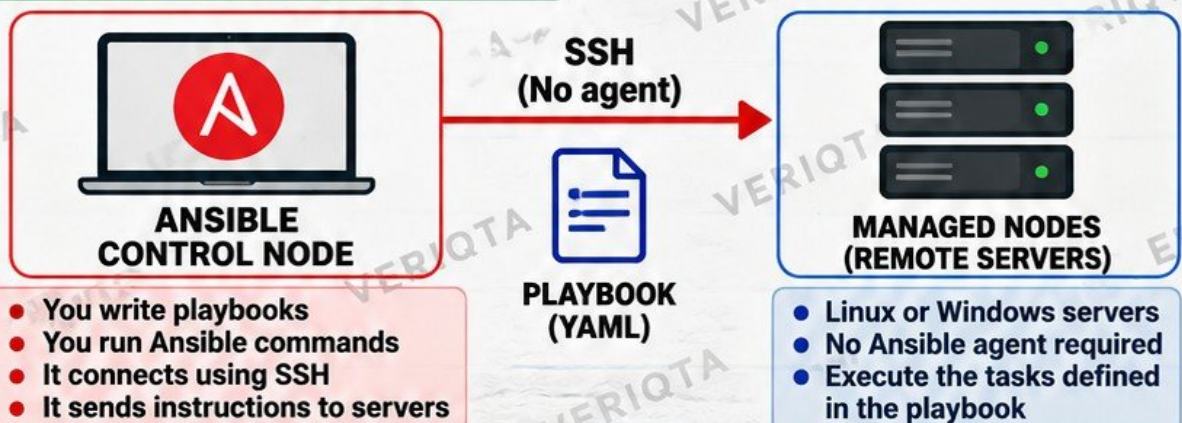
What Problem Does Ansible Solve?

- **Saves time** by automating server setup.
- **Reduces human errors**.
- Keeps environments **consistent** across development, testing and production.
- Makes it easy to manage many servers from **one place**.

Where Ansible Fits in DevOps

- Ansible is used throughout the DevOps lifecycle to:
 - Set up infrastructure.
 - Configure servers and applications.
 - Deploy applications.
 - Manage environments (dev, test, prod).
 - Support CI/CD pipelines.
 - Maintain consistency and repeatability.

How Ansible Works (High Level)



Common Use Cases

-  Install software
-  Configure files
-  Manage services
-  Create users
-  Deploy applications

Key Benefits

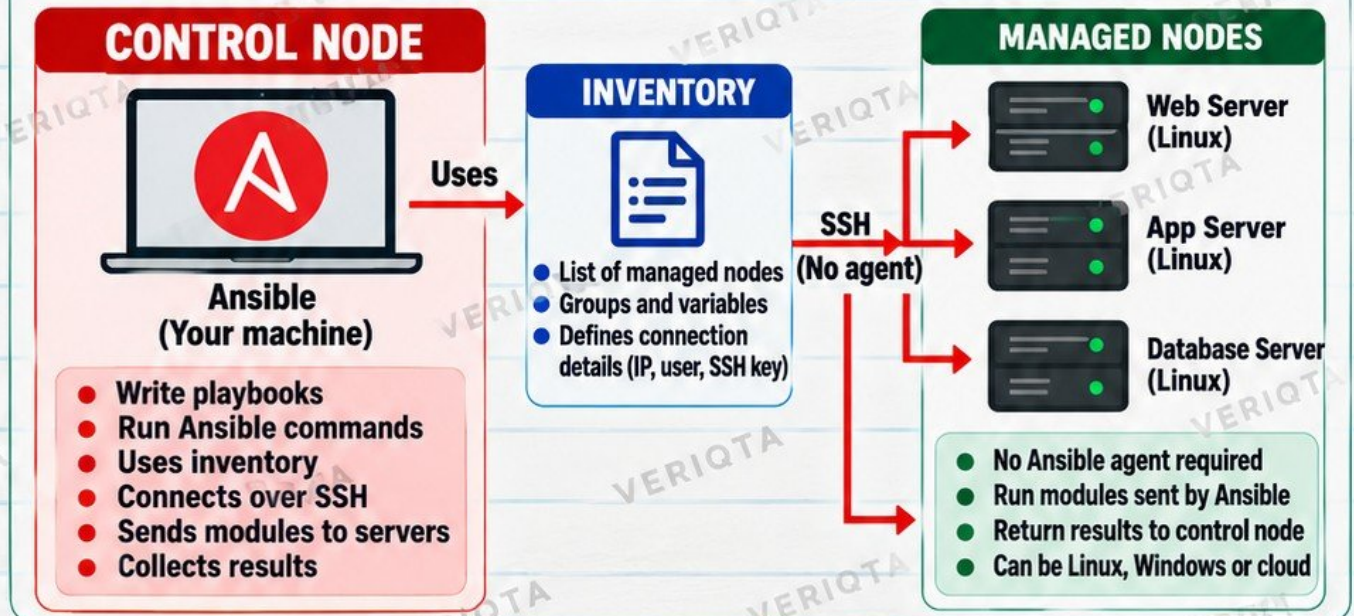
- ✓ Simple and easy to learn
- ✓ Agentless
- ✓ Powerful and flexible
- ✓ Large community support
- ✓ Widely used in real-world DevOps

02 How Ansible Actually Works

Control node, managed nodes, SSH, inventory, modules, and the agentless model

The Big Picture

Ansible runs on a **control node**. You write playbooks or run commands, and Ansible connects to **managed nodes** over **SSH**. It **does not install any agent** on the servers. This makes it simple, secure and easy to manage.



1. Control Node



- This is the machine where Ansible is installed.
- You run playbooks and Ansible commands here.
- It can be your laptop, a bastion server or a CI/CD runner.

2. Inventory



- A file that tells Ansible which servers to manage.
- Can be static (INI or YAML) or dynamic (cloud, tools).
- You can organize servers into groups and set variables.

3. SSH Connection



- Ansible connects to managed nodes using SSH.
- You can use a password or SSH key (recommended).
- No additional software (agent) is installed on the servers.

4. Modules



- Modules are ready-made automation tools in Ansible.
- Each module performs a specific task (e.g. install a package, manage a file, start a service).
- Modules are sent to the remote server, executed and then removed.

5. Managed Nodes



- These are the servers you want to manage.
- They can be physical servers, virtual machines or cloud instances.
- They do not need Ansible installed.
- They only need SSH access and a supported OS.

6. Agentless Model (Why It Matters)



- No agent to install
- Less maintenance
- More secure
- Works with existing servers
- Easier to get started
- Widely used in real-world DevOps environments



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

03 Install Ansible and Verify It

Install Ansible, check ansible --version, understand the environment, and perform first checks

What You Will Do

- Install Ansible on your control node
- Verify the installation
- Check the environment
- Run your first Ansible command
- Make sure everything is working

Prerequisites



A Linux machine (Ubuntu or CentOS/Rocky/AlmaLinux)



Internet access



A user with sudo privileges

1. Install Ansible on Ubuntu/Debian

```
sudo apt update
sudo apt install -y ansible
```



Note:

This installs Ansible from the official Ubuntu/Debian repositories.

2. Install Ansible on CentOS/Rocky/AlmaLinux

```
sudo dnf install -y ansible
# or on older systems use:
sudo yum install -y ansible
```



Note:

This installs Ansible from the official repositories.

3. Verify the Installation

```
ansible --version
```

Example output:

```
ansible [core 2.15.5]
config file = /etc/ansible/ansible.cfg
python version = 3.11.6
jinja version = 3.1.2
executable location = /usr/bin/ansible
```

4. Check Your Environment

```
which ansible
ansible-config dump --only-changed
ansible --version
```



What to check:

- Ansible is installed
- You see the version number
- The correct Python version is used
- The config file path looks correct

5. Run Your First Ansible Command

```
ansible localhost -m ping
```

Expected output:

```
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

6. Troubleshooting Tips

- "command not found" → Ansible is not installed or not in your PATH.
- Permission denied → Use sudo.
- Check Python version → Ansible requires Python 3.
- Network issues → Make sure you have internet access.
- Use `ansible --version` to confirm it is installed correctly.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

04

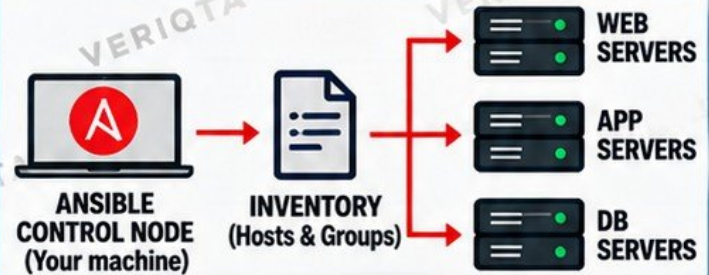
Build Your First Inventory

INI and YAML inventory basics, hosts, groups, aliases, and connection variables

What is an Inventory?

- An inventory is a file that tells Ansible **which servers to manage**.
- It contains **hosts, groups** and **connection variables**.
- You can use INI or YAML format.
- An inventory can be a local file, a dynamic source (like cloud), or a combination.

Inventory Architecture



INI Inventory Example

```

[web]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11

[app]
app1 ansible_host=192.168.1.20
app2 ansible_host=192.168.1.21

[db]
db1 ansible_host=192.168.1.30

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_rsa
  
```

YAML Inventory Example

```

all:
  vars:
    ansible_user: ubuntu
    ansible_ssh_private_key_file: ~/.ssh/id_rsa
  children:
    web:
      hosts:
        web1:
          ansible_host: 192.168.1.10
        web2:
          ansible_host: 192.168.1.11
    app:
      hosts:
        app1:
          ansible_host: 192.168.1.20
        app2:
          ansible_host: 192.168.1.21
    db:
      hosts:
        db1:
          ansible_host: 192.168.1.30
  
```

Common Inventory Elements

Element	Description
Host	A single server (IP or hostname)
Group	A collection of hosts
Alias	A friendly name for a host
Connection variable	Tells Ansible how to connect (e.g. ansible_host, ansible_user)
Group variables	Variables for a group of hosts
Host variables	Variables for a single host
all:vars	Variables applied to all hosts

Useful Commands

```

# List all hosts in the inventory
ansible-inventory -i inventory.ini --list
# Test a specific group
ansible web -i inventory.ini -m ping
# Test the YAML inventory
ansible all -i inventory.yml -m ping
  
```

Best Practices

- Use descriptive group names (web, app, db)
- Use hostnames or aliases instead of only IPs
- Keep sensitive variables in Ansible Vault
- Use group_vars and host_vars for larger environments
- Test your inventory with the ping module



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

05 Your First Ansible Command

Ad hoc commands using ansible, ping, command, shell, and targeting hosts safely

What are Ad Hoc Commands?

- Ad hoc commands are **quick, one-time Ansible commands** you run from the terminal.
- They are useful for testing, troubleshooting and simple tasks.
- They **do not require a playbook**.
- You run them against hosts in your inventory.

Basic Syntax

```
ansible <target> -i <inventory> \
-m <module> -a "<module arguments>"
```

<target> Host(s) or group(s) to run the command on
-i Path to your inventory file
-m Ansible module to use
-a Arguments for the module

1. Test Connectivity with ping

```
ansible all -i inventory.ini -m ping
```

Expected output:

```
web1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

- Checks if Ansible can connect to the hosts.
- Uses the built-in ping module.
- Does not run any system commands.

2. Run a Simple Command

```
ansible all -i inventory.ini -m command -a "uptime"
```

Expected output:

```
web1 | SUCCESS | rc=0 >>
12:34:56 up 1 day, 3:21, 1 user,
load average: 0.00, 0.01, 0.05
```

- Runs a command on the remote server.
- Use the command module for simple commands.
- Does not use a shell, so it is safer.

3. Run a Command with Shell

```
ansible all -i inventory.ini -m shell -a "df -h"
```

Expected output:

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   10G   38G   21% /
...
```

- Runs the command using a shell.
- Use shell when you need shell features like pipes, redirection or environment variables.
- Be careful with user input to avoid security issues.

4. Target Specific Hosts or Groups

```
# Run on a single host
ansible web1 -i inventory.ini -m ping

# Run on a group
ansible web -i inventory.ini -m ping

# Run on multiple hosts
ansible "web:&prod" -i inventory.ini -m ping
```

Common Useful Ad Hoc Commands

Purpose	Command	What It Does
Test connection	ansible all -m ping	Checks connectivity
Check OS	ansible all -m command -a "uname -a"	Shows system info
Check disk space	ansible all -m shell -a "df -h"	Shows disk usage
List users	ansible all -m command -a "whoami"	Shows current user
Check services	ansible all -m shell -a "systemctl status nginx"	Checks a service

Best Practices

- Always test with the ping module first.
- Use the command module for simple tasks.
- Use the shell module only when needed.
- Target specific hosts or groups instead of all.
- Use an inventory file (INI or YAML).
- Use a non-root user and become only when required.
- Check the output for errors.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

06 Your First Playbook

YAML structure, plays, hosts, tasks, modules, become, and running ansible-playbook

What is a Playbook?

- A playbook is a YAML file that tells Ansible what to do on your servers.
- It contains one or more **plays**, each with **hosts**, **tasks** and **modules**.
- You can use a playbook to install software, configure files, manage services and more.
- Playbooks are **idempotent**, so you can run them multiple times safely.

Basic Playbook Structure

```

---
- name: Play name           # Name of the play
  hosts: web                 # Target hosts or group
  become: yes                # Run as sudo (if needed)
  tasks:                     # List of tasks
    - name: Task name        # Name of the task
      ansible.builtin.module: # Module to use
      key: value              # Module options
  
```

Example: Install and Start Nginx

```

---
- name: Install and start Nginx
  hosts: web
  become: yes
  tasks:
    - name: Install Nginx package
      ansible.builtin.apt:
        name: nginx
        state: present
    - name: Start and enable Nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: yes
  
```

Key Elements Explained

Element	Description
---	Start of a YAML file.
name	A human-readable name for the play or task.
hosts	The target hosts or group from your inventory.
become	Run tasks with sudo privileges.
tasks	A list of tasks to execute.
module	The action to perform (e.g. apt, service, copy).
state	Desired state (present, started, etc).
enabled	Keep the service running after reboot.

Common Modules for Beginners

				
apt/yum	service	copy	file	command
Install packages	Manage services	Copy files	Create files/dirs	Run simple commands

Run the Playbook

```
ansible-playbook -i inventory.ini site.yml
```

- **-i** specifies your inventory file.
- **site.yml** is the name of your playbook.
- Ansible connects to the target hosts and runs all the tasks in order.

Important Notes



- Use **become: yes** when you need sudo.
- Always test on a non-production server first.
- Use meaningful names for plays and tasks.
- Check the output for errors.

Expected Output (Simplified)

```

PLAY [Install and start Nginx] *****
TASK [Gathering Facts]              ok: [web1]
TASK [Install Nginx package]        changed: [web1]
TASK [Start and enable Nginx]       changed: [web1]
PLAY RECAP *****
web1  : ok=3  changed=2  unreachable=0  failed=0
  
```

Next Steps

- ✓ Try changing the package (e.g. apache2).
- Add more tasks (copy a file, create a directory).
- Run the playbook again to see idempotency (no changes).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

07 Idempotency, Ansible's Core Idea

Desired state, changed vs OK, repeatable automation, and why rerunning a playbook should be safe

What is Idempotency?

- Idempotency means running the same playbook multiple times gives the **same result**.
- If the server is already in the desired state, Ansible makes no changes and shows **OK**.
- If a change is needed, Ansible makes the change and shows **CHANGED**.
- This makes automation safe, predictable and repeatable.

Desired State Model

- You define the desired state (e.g. package installed, service running).
- Ansible checks the current state on the server.
- If the current state matches the desired state → no change (**OK**).
- If the current state is different → Ansible makes the change (**CHANGED**).
- You can run the playbook as many times as needed and get the same final state.

Example: Install a Package (Idempotent)

```
---
- name: Ensure Nginx is installed
  hosts: web
  become: yes
  tasks:
    - name: Install Nginx package
      ansible.builtin.apt:
        name: nginx
        state: present
```

Run the Playbook (First Run)

```
$ ansible-playbook install-nginx.yml
```

```
TASK [Install Nginx package] *****
changed: [web1]
```

- Ansible installed Nginx because it was not present.
- The task shows **CHANGED**.
- This means the server was updated to reach the desired state.

Run the Playbook Again (Second Run)

```
$ ansible-playbook install-nginx.yml
```

```
TASK [Install Nginx package] *****
ok: [web1]
```

- Ansible checks the server and sees Nginx is already installed.
- No changes are made.
- The task shows **OK**.
- This confirms idempotency.

More Examples of Idempotent Tasks

Task	Module	Desired State	Result on 1st Run	Result on 2nd Run
Install package	apt/yum	present	CHANGED	OK
Remove package	apt/yum	absent	CHANGED	OK
Start service	service	started	CHANGED	OK
Stop service	service	stopped	CHANGED	OK
Create file	file	present	CHANGED	OK
Ensure directory	file	directory	CHANGED	OK
Copy file	copy	present	CHANGED	OK

Key Takeaways

- Idempotency is a core feature of Ansible.
- You define the desired state, not the steps.
- Ansible only makes changes when needed.
- Rerunning the playbook is safe.
- Look for **CHANGED** and **OK** in the output.

Best Practices

- Always use the correct module instead of shell/command.
- Use state: present or state: absent.
- Test by running the playbook multiple times.
- Avoid commands that always report a change.
- Keep your playbooks idempotent for safe automation.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

08 Variables and Facts

Variables, precedence basics, vars, group_vars, host_vars, and gathered system facts

What are Variables?

- Variables store values you can reuse in your playbooks.
- They make your automation **flexible, reusable** and **environment-specific**.
- You can define variables in different places (playbook, vars, group_vars, host_vars, etc).
- Ansible uses **variable precedence** to decide which value to use.

Where to Define Variables

Location	Example Use
Playbook vars	Simple, play-specific variables
vars section	Variables inside a play
group_vars	Variables for a group of hosts
host_vars	Variables for a single host
Inventory vars	Variables in the inventory file
Extra vars (-e)	Override variables at runtime

1. Playbook Variables Example

```
---
- name: Use variables in a playbook
  hosts: web
  become: yes
  vars:
    app_name: nginx
    app_port: 80
  tasks:
    - name: Install {{ app_name }}
      ansible.builtin.package:
        name: "{{ app_name }}"
        state: present
```

2. group_vars and host_vars Example

inventory/ group_vars/ web.yml db.yml	inventory/group_vars/web.yml app_name: nginx app_port: 80 env: production
host_vars/ web1.yml web2.yml	inventory/host_vars/web1.yml ansible_host: 192.168.1.10 app_port: 8080 env: staging

3. Using Variables in Tasks

```
- name: Create config file
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  vars:
    listen_port: "{{ app_port }}"
    server_name: "{{ inventory_hostname }}"
```

4. Gathered Facts (System Information)

```
- name: Show some facts
  hosts: web
  tasks:
    - name: Display hostname
      ansible.builtin.debug:
        msg: "Hostname: {{ ansible_hostname }}"
    - name: Display OS
      ansible.builtin.debug:
        msg: "OS: {{ ansible_distribution }}\n{{ ansible_distribution_version }}"
```

5. Common Facts You Can Use

Fact	Description	Example
ansible_hostname	Server hostname	web1
ansible_distribution	OS name	Ubuntu
ansible_distribution_version	OS version	22.04
ansible_os_family	OS family	Debian
ansible_architecture	System architecture	x86_64
ansible_default_ipv4.address	Main IP address	192.168.1.10

6. Variable Precedence (Simplified)

Higher Priority	1	Extra vars (-e)
	2	Task vars
	3	Block vars
	4	Play vars
	5	Role vars
	6	group_vars / host_vars
	7	Inventory vars
	8	Facts (gathered automatically)
Lower Priority	9	Default vars

Key Takeaways

- Use variables to make your playbooks flexible and reusable.
- You can define variables in many places.
- group_vars and host_vars help manage environment-specific values.
- Facts give you information about the target systems.
- Variable precedence determines which value is used.

Best Practices

- Use meaningful variable names.
- Keep sensitive data in Ansible Vault.
- Use group_vars and host_vars for larger environments.
- Use facts to write dynamic and portable playbooks.
- Know the variable precedence to avoid confusion.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

09 Conditions and Loops

when, loops, lists, registered results, and making playbooks react to system state

What are Conditions and Loops?

- Conditions (**when**) let you run tasks only when certain conditions are true.
- **Loops** help you run the same task for multiple items.
- You can use **lists**, **registered results** and **facts** to make your playbooks react to the system state.
- This makes playbooks smarter, more flexible and more efficient.

Common Use Cases

- ✓ Run a task only if a package is not installed.
- ✓ Create multiple users from a list.
- ✓ Configure multiple files or services.
- ✓ Use registered results to make decisions.
- ✓ Loop over items, lists or dictionaries.
- ✓ Make playbooks adapt to different environments based on system facts.

1. Using Conditions (when)

```
- name: Install Nginx if not installed
  ansible.builtin.package:
    name: nginx
    state: present
  when: ansible_facts.packages.nginx
        is not defined
```

- The task runs only when Nginx is not installed.
- when uses a condition to check the system state.
- You can use facts, variables or registered results in conditions.

2. Using Loops with a List

```
- name: Create multiple users
  ansible.builtin.user:
    name: "{{ item }}"
    state: present
  loop:
    - alice
    - bob
    - charlie
```

- Loop runs the same task for each item in the list.
- item refers to the current value in the loop.
- You can loop over packages, users, files, services and more.

3. Using Registered Results

```
- name: Check if a file exists
  ansible.builtin.stat:
    path: /etc/nginx/nginx.conf
  register: nginx_file

- name: Show message based on result
  ansible.builtin.debug:
    msg: "Nginx config exists"
  when: nginx_file.stat.exists
```

- The result of a task is stored in a variable using **register**.
- You can use the registered result in later tasks.
- This is useful for making decisions based on real output.

4. Looping Over a Dictionary

```
- name: Configure multiple services
  ansible.builtin.service:
    name: "{{ item.key }}"
    state: "{{ item.value }}"
  loop:
    nginx: started
    ssh: started
    firewallld: stopped
```

- You can loop over a dictionary (key: value pairs).
- item.key gives the key and item.value gives the value.
- Useful for configuring multiple services or settings.

Key Takeaways

- Use **when** to run tasks only when conditions are true.
- Use **loops** to run the same task for multiple items.
- Use **registered results** to store task output.
- Combine conditions, loops and facts to make playbooks dynamic and environment-aware.

Best Practices

- Use clear and readable conditions.
- Test your conditions with debug first.
- Use meaningful variable names for registered results.
- Be careful with loops and large lists.
- Use facts and registered results to make playbooks idempotent and intelligent.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

10 Files, Directories and Permissions

file, copy, ownership, modes, configuration deployment, and avoiding dangerous permissions

Why File Management Matters?

- Used to create, manage and deploy files and directories on remote servers.
- Helps keep configuration **consistent** across all hosts.
- Sets correct ownership and permissions for **security**.
- Prevents accidental exposure of sensitive files.
- Essential for deploying application configs, scripts and services.

Key Modules for File Management

Module	Purpose
file	Create, remove or manage files and directories (state, owner, mode).
copy	Copy files from control node to remote host.
template	Copy Jinja2 templates with variables.
fetch	Copy files from remote host to control node.
stat	Get file or directory information.
find	Search for files and directories.
assemble	Combine multiple files into one.
lineinfile	Add or update a single line in a file.
blockinfile	Add or update a block of text in a file.

1. Create a Directory

```
- name: Create application directory
  ansible.builtin.file:
    path: /opt/myapp
    state: directory
    owner: appuser
    group: appuser
    mode: '0755'
```

- Creates the directory if it does not exist.
- Sets owner, group and permissions.
- Safe to run multiple times (idempotent).

2. Copy a File

```
- name: Copy configuration file
  ansible.builtin.copy:
    src: files/app.conf
    dest: /etc/myapp/app.conf
    owner: root
    group: root
    mode: '0644'
```

- Copies a file from your control node to the remote server.
- Sets the correct ownership and permissions.
- Only updates the file if content has changed.

3. Deploy a Template

```
- name: Deploy configuration template
  ansible.builtin.template:
    src: templates/app.conf.j2
    dest: /etc/myapp/app.conf
    owner: root
    group: root
    mode: '0644'
```

- Uses a Jinja2 template with variables.
- Useful for environment-specific configuration.
- Updates the file only if content changes.

4. Set Permissions and Ownership

```
- name: Set correct permissions
  ansible.builtin.file:
    path: /etc/myapp/app.conf
    owner: root
    group: appuser
    mode: '0640'
```

- Ensures the file has the right owner and group.
- Sets secure permissions (0640 in this example).
- Helps protect sensitive configuration files.

5. Check if a File Exists

```
- name: Check if file exists
  ansible.builtin.stat:
    path: /etc/myapp/app.conf
    register: app_conf
```

- Checks if a file or directory exists.
- Stores the result in a variable (app_conf).
- You can use this result with conditions (when).

6. Avoid Dangerous Permissions

Permission	Meaning	Risk
0777	Read, write, execute for everyone	Very dangerous (anyone can modify)
0666	Read and write for everyone	Dangerous (anyone can modify)
0644	Owner read/write, others read	Safe (common for config files)
0755	Owner read/write/execute, others read/execute	Safe (common for directories)
0600	Owner read/write only	Safer (use for sensitive files)

Key Takeaways

- Use file and copy modules to manage files and directories.
- Always set correct ownership and permissions.
- Use templates for dynamic configuration.
- Avoid dangerous permissions like 0777 or 0666.
- Keep your configuration files secure and consistent.

Best Practices

- Use meaningful file and directory names.
- Set the least required permissions.
- Use templates for environment-specific configs.
- Always verify ownership and mode.
- Use the stat module with conditions when needed.
- Keep sensitive files (keys, passwords) with restrictive permissions.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

11 Packages and Services

Install packages, start services, enable boot persistence, and manage application dependencies

Why Package and Service Management Matters?

- Automates software installation and configuration across all servers.
- Keeps application dependencies consistent.
- Manages services (start, stop, restart, enable at boot).
- Helps maintain stable, secure and repeatable environments.
- Reduces manual work and human errors.

Common Package Modules

Module	Purpose	Example Use Case
<code>apt</code>	Manage packages on Debian/Ubuntu	Install nginx, update package cache
<code>yum</code>	Manage packages on RHEL/CentOS 7	Install httpd, install multiple packages
<code>dnf</code>	Manage packages on RHEL/CentOS 8+	Install and manage latest packages
<code>package</code>	Generic module (works on multiple OS)	Use when you don't want OS-specific modules

1. Install a Package (APT - Ubuntu/Debian)

```
- name: Install Nginx package
  ansible.builtin.apt:
    name: nginx
    state: present
    update_cache: yes
```

- Installs the latest version of nginx.
- **update_cache: yes** updates the package index.
- **state: present** ensures the package is installed.

2. Install a Package (YUM/DNF - RHEL/CentOS)

```
- name: Install httpd package
  ansible.builtin.yum:
    name: httpd
    state: present # or use dnf
- name: Install httpd package (DNF)
  ansible.builtin.dnf:
    name: httpd
    state: present
```

- Use **yum** for RHEL/CentOS 7.
- Use **dnf** for RHEL/CentOS 8+.
- Installs httpd (Apache web server).

3. Install Multiple Packages

```
- name: Install multiple packages
  ansible.builtin.package:
    name:
      - nginx
      - git
      - curl
    state: present
```

- Installs multiple packages at once.
- The **package** module works across different OS.
- Useful for application dependencies.

4. Manage a Service

```
- name: Start and enable Nginx service
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: yes
```

- **state: started** ensures the service is running.
- **enabled: yes** makes it start on boot.
- Use **state: stopped** or **restarted** when needed.

5. Check Service Status

```
- name: Check Nginx service status
  ansible.builtin.service_facts:
- name: Debug service status
  ansible.builtin.debug:
    msg: "Nginx status: {{ ansible_facts.services['nginx.service'].state }}"
```

- **service_facts** gathers information about all services.
- You can use the gathered facts to make decisions (when).
- Useful for checking if a service is running before performing actions.

6. Practical Example (Full Task)

```
- name: Install and start Nginx
  ansible.builtin.apt:
    name: nginx
    state: present
    update_cache: yes
- name: Ensure Nginx is running and enabled
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: yes
```

Key Takeaways

- ✓ Use the correct package module for your operating system.
- ✓ Always set the desired state (present/absent).
- ✓ Manage services to ensure they are running and enabled at boot.
- ✓ Keep application dependencies consistent across all servers.
- ✓ Test your tasks to make sure they are idempotent.

Best Practices

- ✓ Use **package** to support multiple operating systems.
- ✓ Always enable critical services at boot.
- ✓ Use meaningful package and service names.
- ✓ Check service status with **service_facts** or **systemd**.
- ✓ Combine package and **service** tasks where appropriate.
- ✓ Keep packages updated for security.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

12 Templates with Jinja2

Turn configuration files into reusable templates using variables and .j2 files

What are Templates with Jinja2?

- Jinja2 is a templating engine used by Ansible to create dynamic files.
- You write a template file with the extension **.j2** and use variables inside it.
- Ansible replaces the variables with actual values and generates the final file.
- Templates help you create reusable, flexible and environment-specific configuration files.

Common Jinja2 Syntax

Syntax	Description
<code>{{ variable }}</code>	Insert a variable value
<code>{% if condition %} ... {% endif %}</code>	Conditional statements
<code>{% for item in list %} ... {% endfor %}</code>	Loop over a list
<code>{{ variable default('value') }}</code>	Default value if not set
<code>{{ variable upper }}</code>	Apply a filter (uppercase)
<code>{{ variable lower }}</code>	Apply a filter (lowercase)
<code>{{ variable length }}</code>	Get length of a value

1. Sample Configuration File

nginx.conf.j2 (template file)

```
user {{ nginx_user }};
worker_processes {{ workers }};

events {
    worker_connections {{ connections }};
}

http {
    include      mime.types;
    default_type application/octet-stream;

    server {
        listen    {{ app_port }};
        server_name {{ server_name }};

        location / {
            root    {{ web_root }};
            index   index.html;
        }
    }
}
```

2. Playbook Using Template

deploy-nginx.yml

```
- name: Deploy Nginx config
  using template
  hosts: web
  become: yes
  vars:
    nginx_user: www-data
    workers: 2
    connections: 1024
    app_port: 80
    server_name: example.com
    web_root: /var/www/html
  tasks:
    - name: Create Nginx config
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: '0644'
        notify: Restart Nginx
  handlers:
    - name: Restart Nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

3. Variables Example

group_vars/web.yml

```
nginx_user: www-data
workers: 4
connections: 2048
app_port: 80
server_name: myapp.com
web_root: /var/www/html
```

4. Rendered Output

/etc/nginx/nginx.conf (final file)

```
user www-data;
worker_processes 4;

events {
    worker_connections 2048;
}

http {
    include      mime.types;
    default_type application/octet-stream;

    server {
        listen 80;
        server_name myapp.com;

        location / {
            root    /var/www/html;
            index   index.html;
        }
    }
}
```

5. Useful Jinja2 Features

Feature	Example
Condition	<code>{% if app_port == 80 %} ... {% endif %}</code>
Loop	<code>{% for item in packages %} ... {% endfor %}</code>
Default value	<code>{{ app_port default(8080) }}</code>
Uppercase	<code>{{ server_name upper }}</code>
Lowercase	<code>{{ server_name lower }}</code>
Length	<code>{{ packages length }}</code>
List join	<code>{{ packages join(' ', ' ') }}</code>

6. Example: Loop in Template

servers.conf.j2

```
{% for server in app_servers %}
    server {{ server }};
{% endfor %}
```

Variables

```
app_servers:
  - 10.0.1.10
  - 10.0.1.11
  - 10.0.1.12
```

7. Key Takeaways

- ✓ Use .j2 extension for template files.
- ✓ Store templates in a templates/ folder.
- ✓ Use variables to make templates reusable.
- ✓ Templates make configuration management flexible and environment-specific.
- ✓ You can use conditions, loops and filters in Jinja2.
- ✓ The template module generates the final file on the remote server.
- ✓ Templates help you manage complex configuration files easily.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

13 Handlers and Notifications

Restart services only when configuration changes and understand event-driven task execution

What are Handlers and Notifications?

- **Handlers** are special tasks that run **only when notified** by other tasks.
- **Notifications** are sent when a task **makes a change** (changed state).
- They help you restart services **only when configuration files actually change**.
- This makes your playbooks more **efficient, safe and idempotent**.

How It Works

-  A task (like a template or copy) runs and makes a change.
-  The task sends a notification to a handler.
-  The handler runs at the end of the play and restarts the service (only if there was a change).

1. Example Playbook with Handler

```
---
- name: Deploy Nginx with handler
  hosts: web
  become: yes
  tasks:
    - name: Deploy Nginx configuration
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: '0644'
        notify: Restart Nginx
  handlers:
    - name: Restart Nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

2. When Does the Handler Run?

Scenario	Task Result	Handler Runs?
Configuration file is changed	changed	Yes
Configuration file is the same	ok	No
Task fails	failed	No
Multiple tasks notify the same handler	changed	Yes (only once)

3. Multiple Notifications Example

```
- tasks:
  - name: Copy application config
    ansible.builtin.copy:
      src: app.conf
      dest: /etc/myapp/app.conf
      notify: Restart MyApp
  - name: Update environment file
    ansible.builtin.template:
      src: env.j2
      dest: /etc/myapp/env.conf
      notify: Restart MyApp
  handlers:
    - name: Restart MyApp
      ansible.builtin.service:
        name: myapp
        state: restarted
```

4. Useful Modules That Commonly Notify Handlers

Module	Purpose	Example Use Case
template	Deploy config files with variables	Nginx, Apache, app configs
copy	Copy static files	Application configs, scripts
lineinfile	Update specific lines	Update settings in a file
blockinfile	Manage blocks of text	Add config blocks
apt / yum / dnf	Install or update packages	Install and update software
command / shell	Run commands	Usually with creates/when

5. Key Takeaways

- ✓ Handlers run only when notified by a changed task.
- ✓ Use handlers to restart services, reload configs or run post-tasks.
- ✓ This prevents unnecessary service restarts.
- ✓ A handler runs once even if notified multiple times.
- ✓ Use handlers with template, copy, lineinfile, blockinfile and package tasks.
- ✓ Helps make your playbooks efficient, safe and idempotent.

6. Best Practices

- Use clear and meaningful handler names.
- Notify handlers only when a real change happens.
- Keep handlers at the end of the play (or in a role).
- Combine related restarts into a single handler when possible.
- Test your playbook multiple times to confirm idempotency.
- Use handlers for service restart, reload, or reconfiguration tasks.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

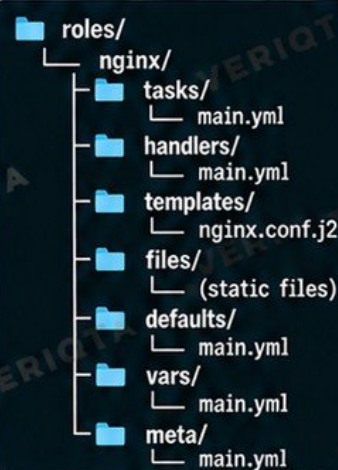
14 Roles and Reusable Automation

Role structure, tasks, handlers, templates, files, defaults, and reusable automation

What are Roles in Ansible?

- Roles are a structured way to organize playbooks for **reusable automation**.
- Each role has a standard directory structure (tasks, handlers, templates, files, defaults, vars and more).
- Roles make your automation **modular, maintainable** and easy to reuse across projects and environments.
- They help you build clean, consistent and **professional** Ansible projects.

Standard Role Structure



- **tasks/** Main automation tasks for the role.
- **handlers/** Tasks that run when notified.
- **templates/** Jinja2 templates (e.g. config files).
- **files/** Static files to copy to the target.
- **defaults/** Default variables with low precedence.
- **vars/** Role-specific variables with higher precedence.
- **meta/** Role metadata and dependencies.

1. Creating a Role

```
ansible-galaxy init nginx
```

- Creates the standard role structure.
- You can then add your tasks, templates, handlers, files and variables.
- Helps you follow best practices and keeps your project organized.

2. Using a Role in a Playbook

```

---
- name: Deploy Nginx using role
  hosts: web
  become: yes
  roles:
    - nginx
  
```

- Include the role under the roles section.
- You can use multiple roles in one playbook.
- Makes your playbooks cleaner and easier to maintain.

3. Example Task (tasks/main.yml)

```

---
- name: Install Nginx
  ansible.builtin.package:
    name: nginx
    state: present
  
```

- Tasks go in `roles/<role>/tasks/`.
- Use modules to define what should be installed or configured.
- Keep tasks focused and idempotent.

4. Example Handler (handlers/main.yml)

```

---
- name: Restart Nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
  
```

- Handlers run only when notified (usually by a changed task).
- Commonly used to restart or reload services.
- Keeps your automation efficient.

5. Example Template (templates/nginx.conf.j2)

```

server {
    listen {{ nginx_port }};
    server_name {{ server_name }};

    root {{ web_root }};
    index index.html;
}
  
```

- Use Jinja2 variables in templates.
- Templates allow you to create dynamic configuration files.
- They are rendered on the target with the actual variable values.

6. Example Defaults (defaults/main.yml)

```

---
nginx_port: 80
server_name: example.com
web_root: /var/www/html
  
```

- Defaults provide default values with low precedence.
- You can override these values in inventory, group_vars or playbook.
- Makes roles more reusable and flexible.

Key Takeaways

- ✓ Roles provide a standard structure for reusable automation.
- ✓ Use tasks, handlers, templates, files, defaults and vars.
- ✓ Keep roles modular, clean and environment-agnostic.
- ✓ Use variables to make roles flexible and reusable.
- ✓ Follow best practices for professional and maintainable projects.

Best Practices

- ✓ Use meaningful role names and structure.
- ✓ Keep tasks small, focused and idempotent.
- ✓ Use handlers to restart services only when needed.
- ✓ Use templates for configuration files.
- ✓ Provide sensible defaults and allow overrides.
- ✓ Test your roles in different environments.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

15 Secrets with Ansible Vault

Protect passwords and sensitive variables, use ansible-vault, and avoid plaintext secrets

Why Use Ansible Vault?

- ✓ Protects sensitive data like passwords, API keys and certificates.
- ✓ Keeps secrets encrypted in your repository.
- ✓ Prevents accidental exposure of confidential information.
- ✓ Works seamlessly with playbooks, roles and variable files.
- ✓ Helps you follow security best practices in real environments.

How Ansible Vault Works



Common Use Cases

- Database passwords
- API keys and tokens
- Cloud credentials (AWS, Azure, GCP)
- TLS/SSL certificates
- Any sensitive configuration values

1. Encrypt a File

```
# Encrypt an existing variables file
ansible-vault encrypt group_vars/
prod/vault.yml
```

- You will be prompted for a vault password.
- The file is encrypted and safe to commit.
- The content is no longer readable.

2. Edit an Encrypted File

```
# Edit the encrypted file
ansible-vault edit group_vars/
prod/vault.yml
```

- Enter your vault password.
- The file opens in your editor decrypted temporarily.
- Changes are automatically re-encrypted when you save and close.

3. View File Content (Optional)

```
# View the content without editing
ansible-vault view group_vars/
prod/vault.yml
```

- Shows the decrypted content in the terminal.
- Useful for quick checks.
- The file remains encrypted on disk.

4. Use Vault File in a Playbook

```
---
- name: Use encrypted variables
  hosts: web
  vars_files:
    - group_vars/prod/vault.yml
  tasks:
    - name: Show database user
      debug:
        msg: "DB User: {{ db_user }}"
```

- Include the encrypted file using vars_files.
- Use the variables just like any other variables.
- You will be asked for the vault password when running the playbook.

5. Run a Playbook with Vault

```
# Option 1: Enter password when prompted
ansible-playbook site.yml

# Option 2: Provide password file
ansible-playbook site.yml \
  --vault-password-file ~/.vault_pass.txt

# Option 3: Set via environment variable
export ANSIBLE_VAULT_PASSWORD_FILE=
~/.vault_pass.txt
ansible-playbook site.yml
```

- Use a secure password (strong and unique).
- A password file avoids typing it every time.
- Keep the password file safe and out of the repository.

6. Encrypt/Decrypt Specific Values

```
# Encrypt a single value
ansible-vault encrypt_string 'mypassword' \
  --name 'db_password'

Result (copy to your vars file):
db_password: !vault |
$ANSIBLE_VAULT;1.1;AES256
333561633538643237616361323736...

# Decrypt a file (if needed)
ansible-vault decrypt group_vars/
prod/vault.yml
```

- Useful when you only need to encrypt one variable.
- You can store the encrypted value directly in a vars file.
- Decrypt only when necessary.

Key Takeaways

- ✓ Never store secrets in plaintext.
- ✓ Use ansible-vault to encrypt files or individual values.
- ✓ Keep your vault password secure.
- ✓ Use vault files with playbooks, roles and variable files.
- ✓ Follow security best practices in real environments.

Best Practices

- ✓ Use a strong and unique vault password.
- ✓ Store vault password files securely and never commit them.
- ✓ Limit access to vault files.
- ✓ Use environment variables or a password file in production.
- ✓ Combine with roles for better organization.
- ✓ Regularly rotate sensitive credentials.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

16 Tags, Check Mode and Safer Changes

--tags, --check, --diff, limiting hosts, and reviewing changes before touching systems

Why Use Tags, Check Mode and Diff?

- ✓ Tags let you run specific parts of a playbook.
- ✓ Check mode (--check) shows what would change without making changes.
- ✓ Diff (--diff) shows the exact difference before applying changes.
- ✓ Limiting hosts helps you test on a few servers first.
- ✓ Together, they make your automation safer and reduce the risk of mistakes in production.

Key Concepts

	Tags	Assign labels to tasks and run only what you need using --tags .
	Check Mode	Simulates changes without actually making them (--check).
	Diff	Shows the exact changes that would be made (--diff).
	Limit Hosts	Run on specific hosts or groups using --limit .
	Safer Changes	Review and validate changes before touching real systems.

1. Example Playbook with Tags

```
---
- name: Configure Nginx with tagged tasks
  hosts: web
  become: yes
  tasks:
    - name: Install Nginx
      ansible.builtin.package:
        name: nginx
        state: present
        tags: [nginx, install]
    - name: Copy Nginx config
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        tags: [nginx, config]
    - name: Restart Nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
        tags: [nginx, restart]
```

Tags allow you to run specific tasks instead of the entire playbook.

2. Run Specific Tags

```
# Run only the install tasks
ansible-playbook site.yml --tags install

# Run multiple tags
ansible-playbook site.yml --tags "install,config"

# Skip specific tags
ansible-playbook site.yml --skip-tags restart
```

- Use **--tags** to run only the tasks you need.
- Use **--skip-tags** to exclude tasks.
- You can combine multiple tags.

3. Check Mode (Dry Run)

```
# See what would change without making changes
ansible-playbook site.yml --check
```

- Runs all tasks in simulation mode.
- Shows which tasks would change (changed) and which would stay the same (ok).
- No actual changes are made to your systems.

4. Diff Mode (See the Exact Changes)

```
# Show the exact changes that would be made
ansible-playbook site.yml --check --diff
```

- Use **--diff** to see the exact difference in files, configs, and settings.
- Very useful for reviewing changes before applying.
- Helps you catch mistakes early.

5. Limit Hosts (Test Safely)

```
# Run on a specific host
ansible-playbook site.yml --limit web1
# Run on a group
ansible-playbook site.yml --limit web
# Run on multiple hosts
ansible-playbook site.yml --limit "web1,web2"
```

- Use **--limit** to run on specific hosts or groups.
- Test on one or a few servers before running on all.
- Reduces the risk of accidental changes in production.

Key Takeaways

- ✓ Use tags to run specific tasks.
- ✓ Use --check to preview changes safely.
- ✓ Use --diff to see exactly what will change.
- ✓ Limit hosts when testing or running changes.
- ✓ These options help you make safer and more controlled changes.

Best Practices

- ✓ Always test with **--check** and **--diff** before applying.
- ✓ Use meaningful and consistent tags.
- ✓ Start with a few hosts using **--limit**.
- ✓ Review the output carefully before running without **--check**.
- ✓ Combine tags, check mode and diff for safer deployments.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

17 Troubleshoot a Failed Playbook

Read failures, use verbosity, inspect SSH problems, undefined variables, permissions, and missing packages

Common Reasons Playbooks Fail

- ❗ SSH connection problems
- ❗ Incorrect inventory or host names
- ❗ Undefined variables
- ❗ Permission/sudo issues
- ❗ Missing packages or repositories
- ❗ Module or task configuration errors
- ❗ Syntax errors in YAML
- ❗ File and path issues (wrong paths, missing files)
- ❗ Service or application configuration issues

Use Verbosity for More Details

```
# Increase verbosity to see more details
ansible-playbook site.yml -v      # normal
ansible-playbook site.yml -vv     # more detail
ansible-playbook site.yml -vvv    # even more detail
ansible-playbook site.yml -vvvv   # full debug
```

- Verbosity helps you see what is happening
- Shows detailed module output
- Helps identify where and why the failure occurred

1. Read and Understand the Error

```
TASK [Install Nginx] *****
fatal: [web1]: FAILED! => {
  "msg": "No package matching 'nginx' is available"
}
```

- Read the full error message, not just the first line.
- Look at the task name and the exact error.
- The error message usually tells you the real problem.

2. Check SSH Connection Problems

```
# Test connection to a host
ansible web1 -m ping

# Use verbose to see SSH details
ansible web1 -m ping -vvv
```

Common SSH issues:

- Wrong host or IP address
- Firewall blocking port 22
- Incorrect SSH key or password
- User does not exist
- SSH key permissions are wrong

3. Debug Undefined Variables

```
TASK [Show variable] *****
fatal: [web1]: FAILED! => {
  "msg": "'app_port' is undefined"
}
```

- Check if the variable is defined in vars, group_vars, host_vars or inventory.
- Use debug to print variable values.
- Make sure variable names are spelled correctly.

4. Fix Permission and Sudo Issues

```
TASK [Install package] *****
fatal: [web1]: FAILED! => {
  "msg": "Permission denied (publickey)."
}
```

- Use the correct user with sudo privileges.
- Make sure sudo is installed and configured.
- Check SSH key permissions (chmod 600 ~/.ssh/id_rsa).
- Verify become: yes and become_user: root if needed.

5. Troubleshoot Missing Packages

```
TASK [Install Nginx] *****
fatal: [web1]: FAILED! => {
  "msg": "No package matching 'nginx' is available"
}
```

- Check if the package name is correct for your OS.
- Ensure the required repository is enabled.
- Update package cache (apt update / yum makecache).
- Use the right module (apt for Debian/Ubuntu, yum/dnf for RHEL).

6. Useful Debugging Tools and Commands

Command	Purpose
ansible web1 -m ping	Test connection to a host
ansible web1 -a "whoami"	Check remote user
ansible-playbook site.yml --vvv	Run with high verbosity
ansible-playbook site.yml --syntax-check	Check playbook syntax
ansible-inventory -i inventory.yml --list	View inventory details
ansible web1 -m setup	View gathered facts
ansible-playbook site.yml --check	Run in check mode (dry run)
ansible-playbook site.yml --limit web1	Run on a specific host

Key Takeaways

- ✓ Always read the full error message.
- ✓ Use verbosity to get more details.
- ✓ Check SSH, variables, permissions and packages first.
- ✓ Use built-in Ansible commands to debug.
- ✓ Test changes safely with **--check** and **--limit**.

Best Practices

- ✓ Write clear and meaningful task names.
- ✓ Validate your playbook with **--syntax-check**.
- ✓ Use **--check** before running on production.
- ✓ Keep your inventory, variables and files well organized.
- ✓ Fix one issue at a time and re-run the playbook.
- ✓ Use logs and verbose output to understand failures.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

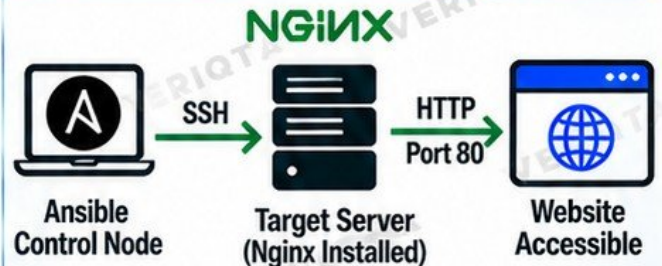
18 Automate a Web Server

Use a playbook to install Nginx, deploy content, configure the service, and verify HTTP availability

What You Will Learn

- ✓ Install Nginx using Ansible.
- ✓ Deploy website content.
- ✓ Configure the Nginx service.
- ✓ Enable the service on boot.
- ✓ Verify HTTP availability.
- ✓ Use handlers, templates and modules.
- ✓ Follow best practices for a reliable automation workflow.

Final Result



1. Playbook to Install and Configure Nginx

```

---
- name: Automate Nginx Web Server
  hosts: web
  become: yes
  tasks:
    - name: Install Nginx
      ansible.builtin.package:
        name: nginx
        state: present
    - name: Deploy website content
      ansible.builtin.copy:
        src: ./index.html
        dest: /var/www/html/index.html
        mode: '0644'
    - name: Start and enable Nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: yes
  
```

This playbook installs Nginx, deploys a simple website and ensures the service is running and enabled on boot.

2. Website Content (index.html)

```

<!DOCTYPE html>
<html>
<head>
  <title>Welcome to VERIQTA</title>
</head>
<body>
  <h1>Nginx Automated with Ansible</h1>
  <p>This website was deployed using Ansible</p>
</body>
</html>
  
```

3. Verify HTTP Availability

```

# Check Nginx service status
ansible web -m shell -a "systemctl status nginx"

# Test HTTP locally or from your browser
curl http://<server-ip>

# Expected output
<h1>Nginx Automated with Ansible</h1>
<p>This website was deployed using Ansible</p>
  
```

4. Useful Ansible Modules Used

Module	Purpose
package	Install Nginx package
copy	Deploy website files
template	Deploy configuration files (if needed)
service	Start, stop and enable the service
file	Manage directories and file permissions
uri	Check HTTP availability
systemd	Manage systemd services (alternative)

5. Optional: Use a Template for Nginx Config

```

- name: Deploy Nginx configuration
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    notify: Restart Nginx
  
```

Key Takeaways

- ✓ You can fully automate a web server with a single playbook.
- ✓ Ansible ensures consistency and repeatability.
- ✓ Handlers can be used to restart Nginx when config changes.
- ✓ Always verify the service and HTTP response.
- ✓ You can extend this to deploy real applications.

Best Practices

- ✓ Use templates for configuration files.
- ✓ Keep website files in version control.
- ✓ Ensure the service is enabled on boot.
- ✓ Verify HTTP availability after deployment.
- ✓ Use meaningful task names and handlers.
- ✓ Add error handling for production use.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

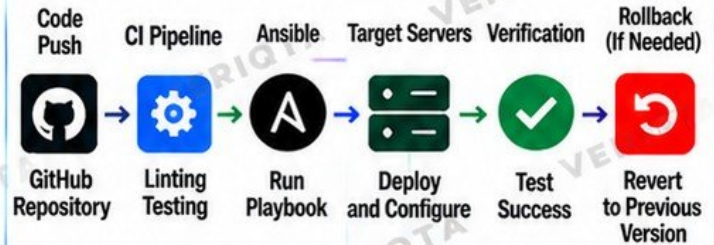
19 Ansible in CI/CD

Repository to pipeline to Ansible to servers, linting, testing, deployment, verification, and rollback thinking

What You Will Learn

- ✓ Integrate Ansible into a CI/CD pipeline.
- ✓ Run linting and basic tests.
- ✓ Deploy changes automatically to servers.
- ✓ Verify the deployment after execution.
- ✓ Understand rollback strategies.
- ✓ Follow best practices for production-ready automation.

End-to-End CI/CD Flow



1. Sample CI/CD Pipeline (GitHub Actions)

```

name: Ansible CI/CD
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: |
          pip install ansible ansible-lint
      - name: Lint playbooks
        run: ansible-lint
      - name: Run tests
        run: ansible-playbook site.yml --check
      - name: Deploy to servers
        run: ansible-playbook site.yml
      - name: Verify deployment
        run: ./scripts/verify.sh
  
```

This pipeline checks your code, runs linting and tests, deploys using Ansible, and verifies the results.

2. Repository Structure

```

ansible-ci-cd/
├── .github/
│   └── workflows/
│       └── deploy.yml
├── inventories/
│   └── production.ini
├── playbooks/
│   └── site.yml
├── roles/
├── tests/
│   └── test.yml
├── scripts/
│   └── verify.sh
└── README.md
  
```

- GitHub Actions runs the pipeline.
- Linting checks code quality.
- Tests validate playbooks.
- Ansible deploys to servers.
- Verification confirms success.
- Rollback is planned for safe releases.

3. Linting and Testing

```

# Lint playbooks
ansible-lint

# Check syntax
ansible-playbook site.yml --syntax-check

# Dry run (no changes)
ansible-playbook site.yml --check
  
```

- `ansible-lint` checks for best practices and errors.
- `--syntax-check` validates YAML syntax.
- `--check` shows what would change without making changes.

4. Verification After Deployment

```

# Check service status
ansible all -m shell -a "systemctl status nginx"

# Test HTTP response
ansible all -m uri -a "url=http://{ansible_host} status_code=200"

# Or use a custom script
./scripts/verify.sh
  
```

- Verify that services are running.
- Test application endpoints (e.g. HTTP).
- Use custom scripts for deeper checks.
- Ensure deployment meets expectations.

5. Rollback Strategy (Example)

```

# Keep a backup of the previous version
ansible all -m copy -a "src=/opt/app/releases/previous dest=/opt/app/current remote_src=yes"

# Or redeploy a specific version/tag
ansible-playbook site.yml -e "app_version=1.0.0"
  
```

- Keep previous releases.
- Redeploy a specific version if needed.
- Use versioning for safe rollbacks.
- Test after rollback to confirm stability.

6. Key Takeaways

- ✓ Integrate Ansible into your CI/CD pipeline for consistent deployments.
- ✓ Use linting and testing to catch issues early.
- ✓ Verify deployments to ensure everything works.
- ✓ Have a rollback plan before making production changes.
- ✓ Keep your repository organized and version controlled.
- ✓ Follow best practices for reliable and repeatable automation.

Best Practices

- ✓ Keep playbooks modular and reusable.
- ✓ Use environments (dev, staging, production).
- ✓ Always run linting and tests before deployment.
- ✓ Verify application health after deployment.
- ✓ Store sensitive data using Ansible Vault.
- ✓ Implement a clear rollback strategy.
- ✓ Use meaningful commit messages.
- ✓ Monitor your deployments.
- ✓ Keep pipelines simple and well documented.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

20 Final Mini Project, Configure Multiple Servers

Build inventory, role, variables, template, handler, Vault-protected data, deployment, and verification

Project Goal

- ✓ Configure multiple servers using Ansible.
- ✓ Build a clean and reusable role.
- ✓ Use variables, templates, handlers.
- ✓ Protect sensitive data with Ansible Vault.
- ✓ Deploy and verify the desired state.
- ✓ Follow real-world best practices.

2. Role Structure

```
roles/webserver/
├── tasks/main.yml
├── handlers/main.yml
├── templates/nginx.conf.j2
├── files/
├── vars/main.yml
├── defaults/main.yml
└── meta/main.yml
```

- Keep your role modular and reusable.
- Use each folder for a specific purpose.
- You can reuse this role for multiple servers.

3. Playbook (Site YAML)

```
---
- name: Configure webserver
  hosts: webserver
  become: yes
  roles:
    - webserver
```

- Target multiple servers using inventory.
- Use roles to keep your playbook clean.
- Run with: `ansible-playbook site.yml`

6. Handler (Restart Service)

```
---
- name: Restart Nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

- Handlers run only when changes happen.
- Use handlers to restart services.
- Keeps your automation efficient.

Key Takeaways

- ✓ Understand the full automation workflow.
- ✓ Use inventory, roles, variables, templates, handlers and Vault.
- ✓ Deploy to multiple servers safely and verify the results.
- ✓ Apply best practices for production-ready automation.

The Learning Flow



1. Inventory (Multiple Servers)

```
[webserver]
web1 ansible_host=192.168.56.10
web2 ansible_host=192.168.56.11

[dbserver]
db1 ansible_host=192.168.56.12
db2 ansible_host=192.168.56.13
```

- Organize servers into groups.
- Use meaningful group names.
- Store in `inventory.ini`.

4. Variables (group_vars)

```
# group_vars/webserver.yml
nginx_port: 80
nginx_server_name: veriqta.local
welcome_message: "Welcome to VERIQTA"
```

- Use `group_vars` or `host_vars`.
- Store configuration values here.
- Make your templates dynamic.

5. Template (Jinja2)

```
server {
    listen {{ nginx_port }};
    server_name {{ nginx_server_name }};

    location / {
        return 200 "{{ welcome_message }}";
    }
}
```

- Use `.j2` files for dynamic configuration.
- Replace hardcoded values with variables.
- Templates make your setup flexible.

7. Vault-Protected Data

```
# Create an encrypted file
ansible-vault create group_vars/
webserver/vault.yml

# Use the variable in your role/playbook
nginx_admin_password: !vault |
$ANSIBLE_VAULT;1.1;AES256....
```

- Protect sensitive information.
- Use `ansible-vault` to encrypt files.
- Never store passwords in plaintext.

8. Verification

```
# Check service status
ansible webserver -m shell -a
"systemctl status nginx"

# Test HTTP response
ansible webserver -m uri -a
"url=http://{{ inventory_hostname }}"
```

- Verify the service is running.
- Check HTTP response from each server.
- Confirm the desired state across all servers.

Best Practices

- ✓ Keep your code modular and reusable.
- ✓ Use meaningful names and folder structure.
- ✓ Protect sensitive data with Ansible Vault.
- ✓ Always verify after deployment.
- ✓ Plan for rollback in real environments.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta